



1. Что мы будем проверять

Сейчас я создам небольшой модуль обработки заявок на возврат товара. Внутри него будет чистая бизнес-логика расчёта суммы, сервис создания возврата и простой repository, работающий с SQLite.

Я специально сделаю так, чтобы в исходной реализации находился один пограничный дефект. Hermes не будет заранее знать, где именно находится ошибка. Сначала агент должен изучить контракт, затем сгенерировать тесты на основании требований и реально запустить их.

Если тест упадёт, мы не будем автоматически переписывать сам тест только ради зелёного результата. Сначала агент должен установить, противоречит ли тест требованиям или проблема находится в production-коде.

После исправления мы отдельно измерим statement и branch coverage. Затем намеренно изменим работающий код и проверим, способны ли созданные тесты обнаружить это изменение.

2. Открываем наш проект в Hermes

Я запускаю Hermes Desktop.

Открываю наш предыдущий Project:

```
dependency-lab
```

Открываю встроенный Terminal сочетанием:

```
Ctrl + `
```

Открываю корневой каталог клонированного проекта и сохраняю его путь:

```
$projectRoot = (Get-Location).Path
```

Команда должна указывать на каталог, содержащий `AGENTS.md`, `requirements.txt` и `.venv`.

Теперь проверяю Python:

```
.\.venv\Scripts\python.exe --version
```

3. Проверяем исходное состояние Git

Перед практикой я хочу понимать исходное состояние отслеживаемых файлов.

Выполняю:

```
git status --short
```

Несвязанные локальные материалы не удаляю и не добавляю в commit. Для этой практики отслеживаю только изменения модуля, тестов, dev-зависимостей и настроек игнорирования артефактов.

Это понадобится позже, когда мы будем делать намеренную мутацию production-кода и затем возвращать исходную реализацию.

4. Устанавливаем инструменты тестирования

Для тестов я буду использовать `pytest`, а для оценки покрытия — `coverage.py`. Добавляю их в `dev-requirements.txt` рядом с другими инструментами разработки:

```
pytest  
coverage
```

Устанавливаю весь набор dev-зависимостей:

```
.\.venv\Scripts\python.exe -m pip install -r dev-requirements.txt
```

После установки проверяю:

```
.\.venv\Scripts\python.exe -m pytest --version
```

Затем:

```
.\.venv\Scripts\python.exe -m coverage --version
```

`coverage.py` умеет измерять не только выполнение строк, но и branch coverage. При использовании `--branch` он учитывает, были ли реально пройдены различные направления условных переходов.

Добавляю служебные результаты тестирования в `.gitignore`:

```
.pytest_cache/  
.coverage  
htmlcov/
```

Удалять уже существующие кеша для этого не требуется.

5. Создаём учебный модуль

Теперь я создаю отдельный каталог:

```
New-Item -ItemType Directory -Force testing_lab
```

Создаю:

```
testing_lab\__init__.py
```

Файл оставляю пустым.

Теперь создаю:

```
testing_lab\refunds.py
```

со следующим кодом:

```
from __future__ import annotations

import sqlite3
from datetime import date
from decimal import Decimal

def calculate_refund(
    items: list[dict],
    requested: dict[int, int],
) -> Decimal:
    """Рассчитывает сумму возврата по выбранным позициям."""

    if not requested:
        raise ValueError("Не выбраны позиции для возврата")

    items_by_product = {item["product_id"]: item for item in items}

    total = Decimal("0")

    for product_id, requested_quantity in requested.items():
        if product_id not in items_by_product:
            raise ValueError("Товар отсутствует в заказе")

        if requested_quantity <= 0:
            raise ValueError("Количество должно быть больше нуля")

        item = items_by_product[product_id]

        if requested_quantity > item["quantity"]:
            raise ValueError("Нельзя вернуть больше товара, чем было куплено")

        unit_price = Decimal(str(item["unit_price"]))
```

```
discount_percent = Decimal(
    str(
        item.get(
            "discount_percent",
            0,
        )
    )
)

paid_unit_price = (
    unit_price * (Decimal("100") - discount_percent) / Decimal("100")
)

total += paid_unit_price * requested_quantity

return total.quantize(Decimal("0.01"))
```

```
class ReturnRepository:
    """Хранилище заявок на возврат."""

    def __init__(
        self,
        connection: sqlite3.Connection,
    ) -> None:
        self.connection = connection

        self.connection.execute(
            """
            CREATE TABLE IF NOT EXISTS returns (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                order_id INTEGER NOT NULL,
                amount TEXT NOT NULL,
                status TEXT NOT NULL
            )
            """
        )

        self.connection.commit()

    def create(
        self,
        order_id: int,
        amount: Decimal,
    ) -> dict:
        cursor = self.connection.execute(
            """
            INSERT INTO returns (
                order_id,
                amount,
            """
```

```
        status
    )
    VALUES (?, ?, ?)
    """
    (
        order_id,
        str(amount),
        "requested",
    ),
)

self.connection.commit()

return self.get(cursor.lastrowid)

def get(
    self,
    return_id: int,
) -> dict | None:
    cursor = self.connection.execute(
        """
        SELECT
            id,
            order_id,
            amount,
            status
        FROM returns
        WHERE id = ?
        """
        (return_id,),
    )

    row = cursor.fetchone()

    if row is None:
        return None

    return {
        "id": row[0],
        "order_id": row[1],
        "amount": Decimal(row[2]),
        "status": row[3],
    }

class RefundService:
    """Создаёт заявку на возврат."""

    def __init__(
        self,
        repository: ReturnRepository,
```

```
) -> None:
    self.repository = repository

def create_request(
    self,
    order: dict,
    requested: dict[int, int],
    today: date,
) -> dict:
    if order["status"] != "delivered":
        raise ValueError("Возврат доступен только для полученного заказа")

    delivered_at = order.get("delivered_at")

    if delivered_at is None:
        raise ValueError("Дата получения заказа неизвестна")

    days_since_delivery = (today - delivered_at).days

    if days_since_delivery < 0:
        raise ValueError("Дата получения не может быть в будущем")

    if days_since_delivery >= 14:
        raise ValueError("Срок возврата истёк")

    amount = calculate_refund(
        order["items"],
        requested,
    )

    return self.repository.create(
        order_id=order["id"],
        amount=amount,
    )
```

Я пока не обсуждаю, где здесь находится намеренный дефект.

6. Создаём отдельный контракт

Теперь создаю:

```
testing_lab\CONTRACT.md
```

Содержимое:

```
# Контракт модуля возвратов
```

Модуль реализует создание заявки на возврат товара.

Расчёт суммы

Пользователь должен выбрать хотя бы одну позицию.

Количество возвращаемого товара должно быть больше нуля.

Нельзя вернуть товар, отсутствующий в заказе.

Нельзя вернуть количество, превышающее купленное.

Разрешено вернуть всё купленное количество позиции.

Сумма рассчитывается по цене товара на момент заказа.

Если для позиции сохранена скидка, возврат рассчитывается по фактически оплаченной цене.

Результат должен иметь точность до двух знаков после запятой.

Создание заявки

Заявку можно создать только для заказа со статусом delivered.

Поле delivered_at обязательно.

Возврат разрешён не позднее чем через 14 календарных дней после delivered_at.

Следовательно, ровно на 14-й день возврат всё ещё разрешён.

Если текущая дата находится раньше delivered_at, данные считаются некорректными.

Сохранение

После успешного создания заявка должна быть сохранена через ReturnRepository.

Начальный статус новой заявки:

requested

Сохранённая сумма должна совпадать с результатом calculate_refund.

Вот этот файл для нас важнее текущей реализации.

Если код и `CONTRACT.md` противоречат друг другу, ожидаемое поведение определяется контрактом.

7. Проверяем baseline

До генерации тестов просматриваю только подготовленные файлы модуля:

```
git diff -- testing_lab
```

Отдельный commit пока не нужен. Единую контрольную точку практики я создам после генерации тестов, исправления дефекта и проверки покрытия.

8. Открываем новую сессию Hermes

В Hermes Desktop создаю новый Chat.

Нажимаю:

```
Ctrl + N
```

Теперь хочу подключить встроенную процедуру TDD.

В поле сообщения ввожу:

```
/test-driven-development
```

Hermes устанавливает bundled Skills как slash-команды, а `test-driven-development` относится к стандартным software-development Skills. Его задача — удерживать цикл RED → GREEN → REFACTOR и не скрывать падающие проверки преждевременным изменением теста.

Если команда не распознаётся, сначала проверяю:

```
/skills
```

или в Terminal:

```
hermes skills list
```

9. Первый промпт: только анализ тестируемого контракта

Я не прошу Hermes немедленно генерировать тесты.

Сначала хочу убедиться, что агент правильно понял задачу.

Отправляю первый реальный промпт:

```
Работаем с учебным модулем testing_lab/refunds.py.
```

```
Источником ожидаемого поведения является testing_lab/CONTRACT.md.
```

Сначала только проанализируй модуль и контракт.

Ничего не изменяй и не создавай тесты.

Определи:

какие части подходят для unit-тестирования;

какие взаимодействия имеет смысл проверить integration-тестом;

какие граничные случаи прямо следуют из CONTRACT.md;

какие фикстуры понадобятся.

Очень важно:

не считай текущую реализацию автоматически правильной;

не выводди ожидаемое поведение только из кода;

если код противоречит CONTRACT.md, контракт имеет приоритет.

После анализа остановись.

Теперь я смотрю, что предложил агент.

Я ожидаю, что он отделит `calculate_refund()` как хороший объект unit-тестирования, а `RefundService` + `ReturnRepository` + `SQLite` — как подходящий небольшой integration-сценарий.

10. Проверяем план тестов

До генерации файлов я проговариваю ожидаемые классы сценариев.

Для `calculate_refund()` нам необходим обычный успешный возврат, применение скидки, полный возврат купленного количества, пустой запрос, неизвестный товар, нулевое или отрицательное количество и превышение купленного количества.

Для `RefundService` важны неправильный статус, отсутствие `delivered_at`, будущая дата, возврат раньше ограничения, а особенно ровно 14-й день.

Для integration-теста я хочу проверить реальное взаимодействие:

RefundService

с:

ReturnRepository

и настоящей SQLite in-memory базой.

Здесь repository мы mock-объектом заменять не будем.

11. Второй промпт: генерируем тесты

Теперь отправляю основной промпт, которым действительно пользуемся:

Теперь сгенерируй тесты для модуля `testing_lab/refunds.py`.

Используй `pytest`.

Ожидаемое поведение бери только из `testing_lab/CONTRACT.md`.

Создай:

`tests/testing_lab/conftest.py`

`tests/testing_lab/test_refunds_unit.py`

`tests/testing_lab/test_refunds_integration.py`

В `conftest.py` создай переиспользуемые `pytest fixtures` для:

типового доставленного заказа;

набора позиций заказа;

SQLite in-memory connection;

`ReturnRepository`.

Для unit-тестов `calculate_refund` обязательно покрой:

обычный возврат одной позиции;

частичный возврат;

возврат всего купленного количества;

расчёт позиции со скидкой;

несколько возвращаемых позиций;

пустой `requested`;

товар, которого нет в заказе;

`quantity = 0`;

отрицательный quantity;

количество больше купленного.

Для RefundService unit-тестами проверь:

заказ не delivered;

отсутствующий delivered_at;

дату получения в будущем;

обычный допустимый срок;

ровно 14-й день после получения.

Для unit-тестов RefundService можешь использовать mock repository, чтобы проверять только бизнес-логику сервиса.

Для integration-теста не используй mock repository.

Создай настоящую sqlite3.connect(":memory:") и проверь полный сценарий:

RefundService

→ calculate_refund

→ ReturnRepository

→ запись в SQLite

→ чтение сохранённой заявки.

Проверь, что сохраняются:

order_id;

amount;

status = "requested".

Не изменяй testing_lab/refunds.py.

Не исправляй production-код даже если тест обнаружит проблему.

После создания файлов запусти:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Верни фактический результат выполнения.

Если какой-либо тест упал, остановись после анализа причины.

Не меняй тест только ради зелёного результата.

Вот здесь Hermes уже получает право создавать тестовые файлы и запускать их.

12. Что такое generated fixture в этой практике

После выполнения открываю в File Browser:

```
tests
testing_lab
conftest.py
```

Я ожидаю увидеть конструкции примерно такого типа:

```
import sqlite3
from datetime import date
from decimal import Decimal

import pytest

from testing_lab.refunds import ReturnRepository

@pytest.fixture
def order_items():
    return [
        {
            "product_id": 101,
            "quantity": 2,
            "unit_price": Decimal("100.00"),
            "discount_percent": Decimal("10"),
        },
        {
            "product_id": 202,
            "quantity": 3,
            "unit_price": Decimal("50.00"),
            "discount_percent": Decimal("0"),
        },
    ]

@pytest.fixture
def db_connection():
    connection = sqlite3.connect(":memory:")

    yield connection

    connection.close()
```

```
@pytest.fixture
def repository(db_connection):
    return ReturnRepository(db_connection)
```

Конкретный код Hermes может немного отличаться.

Мне здесь важен принцип: fixture создаёт известное воспроизводимое состояние.

Pytest fixtures передаются тестам через аргументы функций, а `yield` позволяет выполнить teardown после теста.

13. Смотрим первый реальный запуск

Hermes должен выполнить:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Я ожидаю, что большинство тестов пройдёт.

Но один тест должен упасть.

С высокой вероятностью его название будет связано с 14-м днём, например:

```
test_refund_is_allowed_exactly_on_day_14
```

Из сообщения pytest будет видно, что production-код выбросил:

```
ValueError: Срок возврата истёк
```

хотя `CONTRACT.md` говорит:

```
ровно на 14-й день возврат всё ещё разрешён
```

Теперь начинается самая важная часть.

Я не говорю модели:

```
Исправь тест.
```

14. Третий промпт: анализируем RED

Отправляю:

Проанализируй только упавший тест.

Не изменяй пока ни production-код, ни тесты.

Сопоставь три вещи:

ожидаемое поведение из testing_lab/CONTRACT.md;

сам упавший тест;

условие в testing_lab/refunds.py.

Ответь:

правильно ли сформулирован тест;

соответствует ли он контракту;

где находится причина падения;

какое минимальное изменение production-кода требуется, если ошибка действительно находится там.

После анализа остановись.

Теперь агент должен увидеть условие:

```
if days_since_delivery >= 14:
```

При значении:

14

оно уже блокирует возврат.

Но контракт говорит «не позднее чем через 14 дней», значит разрешены дни от 0 до 14 включительно.

Правильная граница должна блокировать только:

> 14

15. Четвёртый промпт: исправляем production-дефект

Только после анализа отправляю:

Исправь подтверждённый дефект в `testing_lab/refunds.py`.

Разрешено изменить только условие, отвечающее за окончание 14-дневного периода возврата.

Тесты не изменяй.

Никакого другого рефакторинга не выполняй.

После изменения снова запусти:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Покажи:

какая строка `production`-кода изменилась;

результат `pytest`;

подтверждение, что тест ровно на 14-й день теперь проходит.

Ожидаемое изменение:

```
if days_since_delivery > 14:
```

Теперь все тесты должны пройти.

16. Проверяем результат вручную

В Terminal я сам выполняю:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Не полагаюсь только на сообщение агента.

Я хочу увидеть итог примерно такого типа:

```
passed
```

для всех тестов.

Количество тестов может немного отличаться в зависимости от того, как Hermes разделил сценарии.

Главное — отсутствие ошибок.

17. Фиксируем рабочее состояние перед coverage

Теперь смотрю:

```
git diff
```

В diff должны присутствовать:

```
testing_lab/refunds.py
tests/testing_lab/conftest.py
tests/testing_lab/test_refunds_unit.py
tests/testing_lab/test_refunds_integration.py
```

Проверяю, что Hermes не изменил что-то несвязанное.

Если всё правильно:

```
git add .gitignore dev-requirements.txt testing_lab tests/testing_lab
```

```
git commit -m "Практика 3.6: добавить тестирование модуля возвратов"
```

Теперь у нас есть единая рабочая точка практики, к которой позднее можно вернуться после mutation experiment.

18. Измеряем coverage

Теперь я хочу узнать не просто количество тестов, а какие части модуля реально выполнялись.

Отправляю Hermes следующий промпт:

```
Теперь измерь покрытие только production-модуля:

testing_lab/refunds.py

Тесты находятся в:

tests/testing_lab

Используй coverage.py с branch coverage.

Сначала удали предыдущие данные coverage, если они существуют.

Затем запусти pytest через coverage так, чтобы измерялся пакет testing_lab.

После этого выведи текстовый report с отсутствующими строками.
```

Production-код и тесты пока не изменяй.

Проанализируй:

statement coverage;

branch coverage;

какие строки или ветви остались непроверенными;

какие из пробелов действительно требуют дополнительного теста, а какие могут быть несущественными для текущего контракта.

Агент должен выполнить эквивалент:

```
.\.venv\Scripts\python.exe -m coverage erase
```

Затем:

```
.\.venv\Scripts\python.exe -m coverage run --branch --source=testing_lab -m pytest tests\testing_lab
```

После этого:

```
.\.venv\Scripts\python.exe -m coverage report -m
```

`coverage report` показывает количество исполняемых statements, пропуски и итоговое покрытие, а режим `--branch` дополнительно учитывает ветви условного выполнения.

19. Не просим автоматически добиться 100%

Теперь я объясняю важную вещь.

Если coverage оказался, например:

92%

я не пишу модели:

Сделай 100%.

Сначала мне нужно понять, что именно осталось непроверенным.

Поэтому мы дали агенту команду анализировать missing lines и missing branches.

100% line coverage можно получить даже достаточно слабым набором тестов. Сам процент не является нашей конечной целью.

20. Пятый промпт: закрываем осмысленный пробел

Если Hermes обнаружил непроверенную ветвь, отправляю следующий промпт:

На основании предыдущего coverage report выбери только те непокрытые ветви testing_lab/refunds.py, которые соответствуют реальному поведению из CONTRACT.md.

Если важная ветвь действительно не проверяется, добавь минимальный тест.

Не добавляй бессмысленные тесты только ради увеличения процента coverage.

Production-код не изменяй.

После добавления теста снова запусти:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

а затем повтори branch coverage.

В конце объясни, какую конкретную ветвь теперь способен обнаруживать новый тест.

Так агент получает правильную цель: закрыть поведенческий пробел, а не просто увеличить цифру.

21. Строим HTML coverage report

Теперь я показываю визуальный вариант.

В Terminal выполняю:

```
.\.venv\Scripts\python.exe -m coverage html
```

Coverage создаёт каталог:

```
htmlcov
```

Каталог не добавляется в Git, потому что уже указан в `.gitignore`.

Открываю:

```
htmlcov\index.html
```

в браузере.

Можно выполнить:

```
Start-Process .\htmlcov\index.html
```

В HTML-отчёте я могу открыть:

```
testing_lab/refunds.py
```

и увидеть, какие строки выполнялись, а какие ветви остались частично покрыты. Coverage.py специально отображает частично покрытые branch-линии отдельно.

22. Теперь проверяем integration-тест отдельно

До этого pytest запускал всё вместе.

Теперь я хочу отдельно убедиться, что integration-тест действительно взаимодействует с repository.

В Terminal выполняю:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab\test_refunds_integration.py -v
```

Открываю сам тест.

В нём должна использоваться реальная:

```
sqlite3.connect(":memory:")
```

и настоящий:

```
ReturnRepository
```

Если Hermes вместо repository поставил:

```
Mock()
```

это уже не тот integration-сценарий, который мы требовали.

Такой тест нужно исправить.

23. Проверяем, что integration-тест действительно проверяет сохранение

В integration-тесте я хочу увидеть не только вызов:

```
service.create_request(...)
```

но и проверку того, что данные действительно оказались в repository.

Логика должна быть примерно такой:

```
result = service.create_request(
    order=order,
    requested={101: 1},
    today=today,
)

stored = repository.get(result["id"])

assert stored is not None
assert stored["order_id"] == order["id"]
assert stored["amount"] == result["amount"]
assert stored["status"] == "requested"
```

Тогда тест проверяет границу между сервисом и хранилищем.

24. Проверяем качество unit-тестов через мутацию

Теперь мы знаем, что тесты зелёные.

Но возникает вопрос: способны ли они вообще ловить неправильные изменения?

Я провожу простейший mutation experiment вручную.

Сначала убеждаюсь, что Git чистый:

```
git status --short
```

Если последние тесты были добавлены после предыдущего commit, сначала фиксирую их.

Теперь даю Hermes специальный промпт:

```
Сейчас проводим учебное mutation testing.

Не исправляй тесты.

В testing_lab/refunds.py найди проверку:

requested_quantity > item["quantity"]

и временно измени оператор так, чтобы условие стало:
```

```
requested_quantity >= item["quantity"]
```

Больше ничего не изменяй.

После мутации запусти:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Не исправляй падение.

Сообщи:

какой тест обнаружил мутацию;

какое корректное поведение нарушилось;

считается ли этот мутант обнаруженным тестовым набором.

Теперь мы сознательно портим production-код.

25. Что должна сделать эта мутация

До изменения код разрешал:

```
requested_quantity == purchased_quantity
```

Это правильно, потому что контракт прямо разрешает вернуть всё купленное количество.

После замены:

```
>
```

на:

```
>=
```

полный возврат ошибочно запрещается.

Если ранее сгенерированный тест:

```
возврат всего купленного количества
```

реально существует, он должен упасть.

Это хороший результат.

Мы специально сломали программу, и тест обнаружил изменение.

Такой мутант считается «убитым» тестами.

26. Анализируем mutation result с Hermes

Если тест действительно упал, задаю:

Не изменяй код.

Объясни, почему эта мутация была обнаружена.

Покажи связь между:

правилом из CONTRACT.md;

изменённым условием;

конкретным упавшим тестом.

Ответь, что именно этот тест защищает от регрессии.

Теперь становится видно, что тест нужен не ради coverage.

Он защищает конкретное бизнес-правило:

разрешено вернуть всё купленное количество позиции

27. Возвращаем production-код после мутации

После эксперимента не прошу AI вручную восстанавливать строку.

Использую Git как надёжную контрольную точку.

В Terminal:

```
git restore testing_lab\refunds.py
```

Проверяю:

```
git diff
```

Изменения production-кода должны исчезнуть.

Теперь ещё раз:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab -v
```

Все тесты снова должны пройти.

28. Проверяем более слабый тестовый набор

Теперь можно показать обратную ситуацию.

Если бы Hermes изначально не создал тест для полного количества, такая мутация могла бы остаться незамеченной.

Именно поэтому я не оцениваю suite только количеством функций `test_*`.

Два хорошо подобранных boundary-теста иногда ценнее десяти тестов одного и того же happy path.

29. Отдельный промт для поиска слабых тестов

Теперь даю Hermes ещё один рабочий промт:

Проведи review только тестов из:

tests/testing_lab

Production-код не изменяй.

Для каждого теста определи, какое наблюдаемое поведение он защищает.

Найди тесты, которые:

дублируют друг друга;

проверяют детали реализации вместо внешнего поведения;

имеют слабые assertions;

могут пройти даже при существенной ошибке production-кода.

Не переписывай тесты автоматически.

Верни только анализ и конкретные рекомендации.

Этот промт полезен после генерации.

AI здесь проверяет уже собственный или ранее созданный тестовый код как отдельный артефакт.

30. Проверяем, что тест не вычисляет ожидаемый результат тем же алгоритмом

Я отдельно открываю тест расчёта скидки.

Плохой вариант выглядел бы так:

```
expected = price * (Decimal("100") - discount_percent) / Decimal("100")

assert calculate_refund(...) == expected
```

Если production и тест используют практически один алгоритм, одна и та же логическая ошибка может оказаться в обеих сторонах.

Для небольших известных примеров мне лучше видеть конкретный expected value:

```
assert result == Decimal("90.00")
```

Тогда expected value независим от внутренней реализации функции.

31. Проверяем изоляцию unit-теста сервиса

В unit-тестах `RefundService` repository действительно можно заменить mock-объектом.

Например, я ожидаю что-то вроде:

```
from unittest.mock import Mock

repository = Mock()

repository.create.return_value = {
    "id": 1,
    "order_id": 10,
    "amount": Decimal("90.00"),
    "status": "requested",
}
```

Здесь мы проверяем бизнес-логику самого сервиса.

Но integration-тест затем использует настоящий repository.

Таким образом, два типа тестов не дублируют друг друга, а проверяют разные границы.

32. Финальный промпт для Hermes

После всех изменений я отправляю агенту итоговую проверку:

Проведи финальную проверку тестирования модуля `testing_lab/refunds.py`.

Ничего больше не изменяй.

Последовательно выполни:

1. Все тесты `tests/testing_lab` через `pytest`.
2. Отдельно `integration`-тесты.
3. `Branch coverage` для пакета `testing_lab`.

После этого подготовь итоговый отчёт.

В отчёте раздели:

какое поведение проверяется `unit`-тестами;

какое взаимодействие проверяется `integration`-тестом;

какие `fixtures` используются;

какой фактический результат `pytest`;

какой фактический `coverage`;

какой пограничный `production`-дефект был обнаружен тестами;

какую намеренную мутацию тесты смогли обнаружить.

Не утверждай то, что не подтверждено фактическим запуском команд.

Это последний промпт текущей практики.

Здесь агент уже ничего не создаёт. Он собирает результаты реальных проверок.

33. Смотрим Git diff

В Terminal:

```
git status --short
```

Затем:

```
git diff
```

Если после финальной генерации появились новые тесты, просматриваю их.

Я хочу убедиться, что production-код не был дополнительно изменён без необходимости.

Если после контрольной точки были добавлены осмысленные тесты для непокрытых ветвей, рассматриваю их и включаю в единый commit этой практики. Служебные отчёты coverage в commit не входят.

34. Что именно сгенерировал AI

Теперь я отделяю результат агента от процесса проверки.

Hermes помог нам проанализировать контракт, сформировать сценарии, создать pytest-файлы и fixtures.

Но ценность появилась только после фактического запуска:

```
pytest
```

Затем мы получили вторую обратную связь через:

```
coverage
```

И третью — через намеренную мутацию production-кода.

Именно эта последовательность позволяет оценить тесты намного объективнее, чем простой просмотр сгенерированного кода.

35. Итог практики

В этой работе я не давал Hermes запрос вида:

```
Напиши тесты для refunds.py.
```

Вместо этого мы отделили контракт от реализации и сначала заставили агента понять, какое поведение вообще необходимо проверять.

После этого AI сгенерировал unit-тесты для локальной бизнес-логики и integration-тест для взаимодействия сервиса с реальным SQLite repository.

Сгенерированные fixtures создали контролируемые данные и тестовую базу.

Первый запуск тестов обнаружил настоящий пограничный дефект: production-код запрещал возврат ровно на 14-й день, хотя контракт разрешал такой сценарий.

Мы исправили production-код, а не переписали тест под существующую ошибочную реализацию.

Затем branch coverage показал, какие направления выполнения программы действительно прошли тесты. Coverage здесь использовался как диагностический инструмент, а не как самоцель.

Наконец, мы намеренно изменили оператор сравнения в production-коде. Если тест полного возврата упал, значит набор тестов действительно способен обнаружить определённую регрессию.

В результате AI использовался не просто как генератор файлов `test_*.py`, а как участник полного проверяемого цикла: анализ требований, генерация тестов, фактический запуск, диагностика падения, проверка покрытия и оценка чувствительности тестового набора к неправильному изменению программы.
